Compte rendu du TP2
Réalisé par Meriam Boukraa

Exercice 1:

J'ai commencé par faire le transfert de la dataset vers mon dossier TP2/data:

```
PS C:\Users\meria> wsl
(base) meria@Meriam:/mnt/c/Users/meria$ cd /mnt/c/Users/meria/Downloads/archive
(base) meria@Meriam:/mnt/c/Users/meria/Downloads/archive$ ls -lh cardio_train.csv
-rwxrwxrwx 1 meria meria 2.9M Sep 24 11:38 cardio_train.csv
(base) meria@Meriam:/mnt/c/Users/meria/Downloads/archive$ scp cardio_train.csv tsp-client:~/TP-HPC/TP2/data/
mboukraa@ssh3.imtbs-tsp.eu's password:
mboukraa@157.159.104.128's password:
cardio_train.csv                                              100% 2873KB   8.4MB/s   00:00
(base) meria@Meriam:/mnt/c/Users/meria/Downloads/archive$ ssh tsp-client
mboukraa@ssh3.imtbs-tsp.eu's password:
mboukraa@157.159.104.128's password:

############################################### DE TSP ###############################################

Welcome to the Albator HPC Cluster
Operated by the Direction de l'Enseignement - Télécom SudParis
```

Question 1:

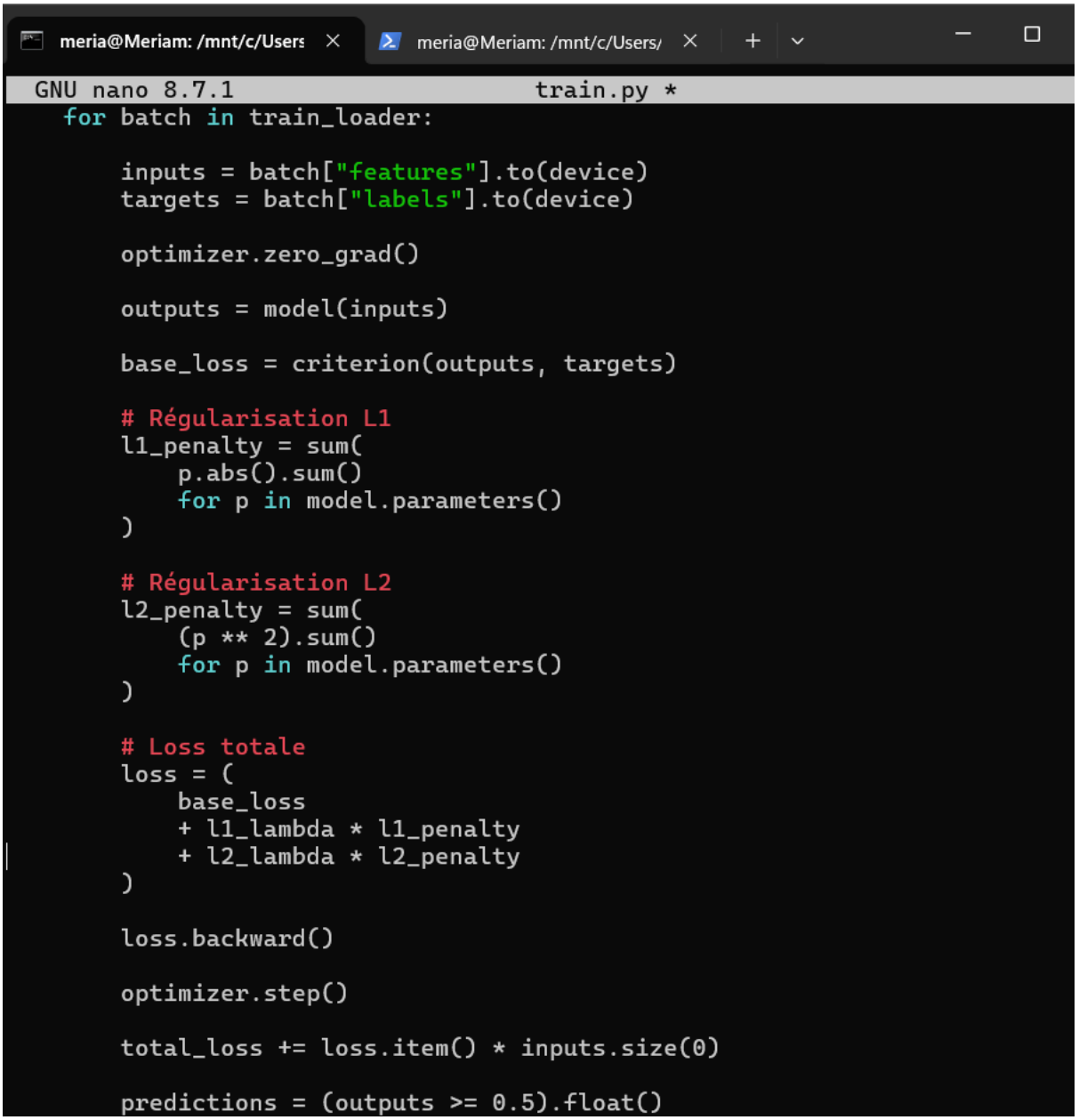
Appliquer StandardScaler sur l'ensemble du dataset avant de réaliser le split entraîne une fuite de données, car les statistiques utilisées pour la normalisation (moyenne et écart-type) sont calculées à partir des données d'entraînement mais aussi des données de validation et de test. Le modèle bénéficie ainsi indirectement d'informations provenant des données qu'il est censé ne pas connaître. Il faut donc effectuer le split en premier, ajuster (fit) le scaler uniquement sur le jeu d'entraînement, puis utiliser ce même scaler pour transformer les jeux de validation et de test.

Question 2:

Pour un dataset trop volumineux pour être chargé entièrement en mémoire, on peut utiliser IterableDataset. Il permet de lire les données progressivement, par exemple par morceaux (chunks), au lieu de charger tout le dataset en RAM.

Exercice 2:

Voici la partie du code contenant des cases à remplir:



Voici une capture d'écran sur le training avant d'effectuer les changement demandés dans les questions:

```
[compute] [job 3402] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP2 (main
)
$ python train.py
Device : cuda
Epoch 1/10 - Loss: 0.8449 - Accuracy: 0.6304
Epoch 2/10 - Loss: 0.8115 - Accuracy: 0.6463
Epoch 3/10 - Loss: 0.7997 - Accuracy: 0.6502
Epoch 4/10 - Loss: 0.7920 - Accuracy: 0.6522
Epoch 5/10 - Loss: 0.7847 - Accuracy: 0.6553
Epoch 6/10 - Loss: 0.7763 - Accuracy: 0.6563
Epoch 7/10 - Loss: 0.7680 - Accuracy: 0.6588
Epoch 8/10 - Loss: 0.7600 - Accuracy: 0.6619
Epoch 9/10 - Loss: 0.7520 - Accuracy: 0.6643
Epoch 10/10 - Loss: 0.7438 - Accuracy: 0.6684
[compute] [job 3402] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP2 (main
)
$
```

Question 1:

Changement de **l1_lambda = 0.1** et **l2_lambda = 0**:

```
)

l1_lambda = 0.1
l2_lambda = 0
```

Résultat:

```
[compute] [job 3402] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP2 (main
)
$ python train.py
Device : cuda
Epoch 1/10 - Loss: 6.7246 - Accuracy: 0.4942
Epoch 2/10 - Loss: 1.6340 - Accuracy: 0.4998
Epoch 3/10 - Loss: 1.6340 - Accuracy: 0.4977
Epoch 4/10 - Loss: 1.6340 - Accuracy: 0.5024
Epoch 5/10 - Loss: 1.6340 - Accuracy: 0.4989
Epoch 6/10 - Loss: 1.6340 - Accuracy: 0.4995
Epoch 7/10 - Loss: 1.6340 - Accuracy: 0.5013
Epoch 8/10 - Loss: 1.6340 - Accuracy: 0.4968
Epoch 9/10 - Loss: 1.6340 - Accuracy: 0.5011
Epoch 10/10 - Loss: 1.6340 - Accuracy: 0.5017
[compute] [job 3402] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP2 (main
)
$
```

Interprétation:

Avec les paramètres initiaux (l1_lambda = 1e-4 et l2_lambda = 1e-3), la loss diminue progressivement de 0,8449 à 0,7438 et la précision augmente de 63,04 % à 66,84 %. Le modèle apprend donc correctement.

En revanche, avec une régularisation L1 forte (l1_lambda = 0,1 et l2_lambda = 0), la loss reste élevée et la précision reste proche de 50 %. Dans notre expérience, la précision passe de 49,42 % à environ 50 % après 10 epochs.

Cette forte pénalisation L1 pousse les poids du réseau vers zéro et limite fortement sa capacité à apprendre les relations présentes dans les données. La régularisation devient alors excessive et provoque un phénomène de sous-apprentissage

Question 2:

Dans PyTorch, l'argument weight_decay de l'optimiseur permet d'appliquer automatiquement une régularisation L2. Par exemple, avec SGD :

```
optimizer = optim.SGD(
    model.parameters(),
    lr=0.01,
    weight_decay=1e-3
)
```

Il n'est donc pas nécessaire de calculer manuellement la pénalité L2 dans la boucle d'entraînement.

Question 3:

La régularisation L1 pénalise la somme des valeurs absolues des poids. Elle tend à rendre certains poids exactement nuls, ce qui produit un réseau plus sparse et peut permettre de sélectionner implicitement les variables les plus importantes.

La régularisation L2 pénalise la somme des carrés des poids. Elle tend à réduire progressivement l'ensemble des poids vers des valeurs plus petites, sans généralement les rendre exactement nuls.

Exercice 3:

Voici la partie du code complétée:

```
# Choix de l'optimiseur
if opt_name == "SGD":
    optimizer = optim.SGD(
        model.parameters(),
        lr=learning_rate
    )

elif opt_name == "Momentum":
    optimizer = optim.SGD(
        model.parameters(),
        lr=learning_rate,
        momentum=0.9
    )

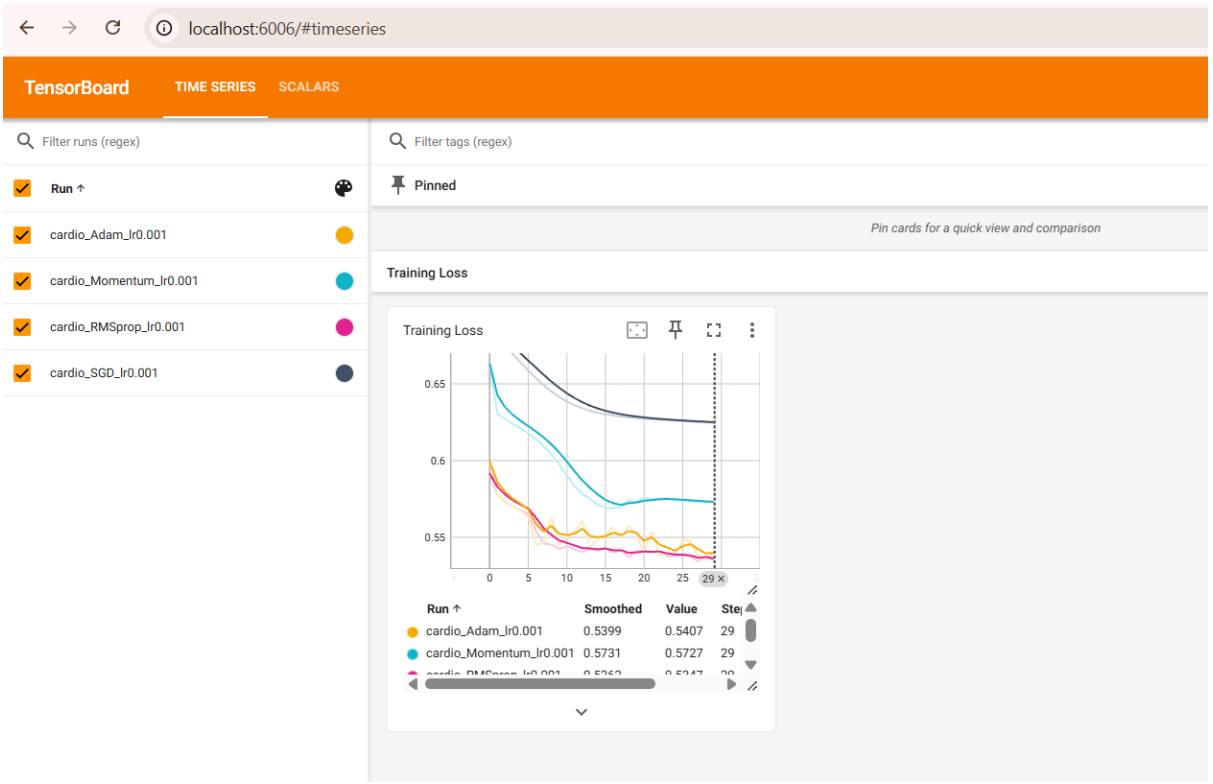
elif opt_name == "RMSprop":
    optimizer = optim.RMSprop(
        model.parameters(),
        lr=learning_rate
    )

elif opt_name == "Adam":
    optimizer = optim.Adam(
        model.parameters(),
        lr=learning_rate
    )

writer = SummaryWriter(
    f"runs/cardio_{opt_name}_lr{learning_rate}"
)
```

Question 1:

Voici la capture d'écran de TensorBoard superposant les courbes de perte des 4 optimiseurs:



Question 2:

Dans notre expérience, Adam et RMSprop présentent la convergence initiale la plus rapide. Dès les premières epochs, leur loss diminue beaucoup plus rapidement que celle du SGD simple. Adam atteint notamment une loss de 0,5449 à l'epoch 7, contre 0,5542 pour RMSprop et 0,6538 pour SGD.

Question 3:

La courbe du Momentum diminue plus rapidement que celle du SGD simple. Dans notre expérience, le SGD passe d'une loss de **0,6863 à 0,6247** en 30 epochs, tandis que le Momentum passe de **0,6633 à 0,5727**.

L'ajout du moment permet de prendre en compte une partie de la direction des mises à jour précédentes. Cela permet d'accélérer la descente de gradient et de progresser plus rapidement vers une zone de minimum.

Exercice 4:

Voici les parties du code qu'on m'a demandé de compléter dans cet exercice:

```
with torch.no_grad():

    for batch in test_loader:

        inputs = batch["features"].to(device)
        targets = batch["labels"].to(device)

        outputs = model(inputs)
```

et:

```
# Calcul des métriques
precision = precision_score(
    all_targets,
    all_preds_classes
)

recall = recall_score(
    all_targets,
    all_preds_classes
)

f1 = f1_score(
    all_targets,
    all_preds_classes
)

auc = roc_auc_score(
    all_targets,
    all_preds_probs
)

print(
    f"Precision: {precision:.4f} | "
    f"Recall: {recall:.4f} | "
    f"F1: {f1:.4f} | "
```

```
^G Help        ^O Write Out   ^F Where Is    ^K Cut
^X Exit        ^R Read File   ^\ Replace     ^U Paste
```

Voici un extrait de l'output reçu après que j'ai lancé: python train.py:

```
================================================
ÉVALUATION SUR L'ENSEMBLE DE TEST
================================================

--- SGD ---
Precision: 0.6533 | Recall: 0.6162 | F1: 0.6342 | AUC: 0.7020

--- Momentum ---
Precision: 0.7455 | Recall: 0.7151 | F1: 0.7300 | AUC: 0.7977

--- RMSprop ---
Precision: 0.7769 | Recall: 0.6632 | F1: 0.7156 | AUC: 0.8018

--- Adam ---
Precision: 0.7648 | Recall: 0.6764 | F1: 0.7179 | AUC: 0.8022

======================================================================
RÉSULTATS FINAUX
======================================================================
SGD         | Precision: 0.6533 | Recall: 0.6162 | F1: 0.6342 | AUC: 0.7020
Momentum    | Precision: 0.7455 | Recall: 0.7151 | F1: 0.7300 | AUC: 0.7977
RMSprop     | Precision: 0.7769 | Recall: 0.6632 | F1: 0.7156 | AUC: 0.8018
Adam        | Precision: 0.7648 | Recall: 0.6764 | F1: 0.7179 | AUC: 0.8022
[compute] [job 3423] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP2 (main)
$
```

Question 1:

La **Precision** mesure la proportion de prédictions positives qui sont réellement positives. Elle est définie par :

Precision = TP / (TP + FP)

où TP représente les vrais positifs et FP les faux positifs.

Le **Recall**, aussi appelé sensibilité, mesure la proportion de cas réellement positifs qui sont correctement détectés par le modèle. Il est défini par :

Recall = TP / (TP + FN)

où FN représente les faux négatifs.

Ainsi, une Precision élevée signifie que le modèle fait peu de fausses alertes parmi ses prédictions positives, tandis qu'un Recall élevé signifie que le modèle détecte une grande partie des cas positifs réels.

Question 2:

Dans le cadre d'un **dépistage médical**, on cherche généralement à avoir un **Recall élevé**, car il est important de détecter le plus grand nombre possible de personnes réellement atteintes.

Un faux négatif correspond à une personne malade que le modèle classe comme non malade. Dans un contexte médical, ce type d'erreur peut être important car la personne risque de ne pas bénéficier d'examens complémentaires ou d'une prise en charge.

À l'inverse, un faux positif correspond à une personne considérée comme potentiellement malade alors qu'elle ne l'est pas. Cela peut entraîner des examens supplémentaires, mais dans un contexte de dépistage, on peut généralement accepter davantage de faux positifs afin de limiter le nombre de cas réellement malades qui ne seraient pas détectés.

Le choix dépend néanmoins du contexte clinique et du coût respectif des faux positifs et des faux négatifs.

Question 3:

La Precision, le Recall et le F1-score sont calculés ici après avoir fixé un **seuil de classification à 0,5**. Ainsi, une probabilité supérieure à 0,5 est considérée comme une prédiction positive, tandis qu'une probabilité inférieure ou égale à 0,5 est considérée comme négative.

L'**AUC (Area Under the ROC Curve)** mesure au contraire la capacité du modèle à distinguer les classes positives et négatives en considérant différents seuils de classification. Elle ne dépend donc pas d'un seuil unique de 0,5.

Dans notre expérience, par exemple, RMSprop obtient une AUC de **0,8018** et Adam une AUC de **0,8022**, alors que leurs Precision et Recall au seuil de 0,5 sont différents. Cela montre que l'AUC permet d'évaluer la capacité générale de discrimination du modèle, tandis que la Precision, le Recall et le F1 évaluent les performances pour un seuil de décision donné.

Une AUC proche de 1 indique une bonne capacité de discrimination entre les deux classes, tandis qu'une AUC proche de 0,5 correspond à une performance proche d'une classification aléatoire.